

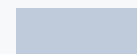
# 答错之后，模型改哪里？

让机器认出你的饮料

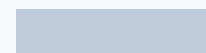


预测分数

可乐



雪碧



芬达



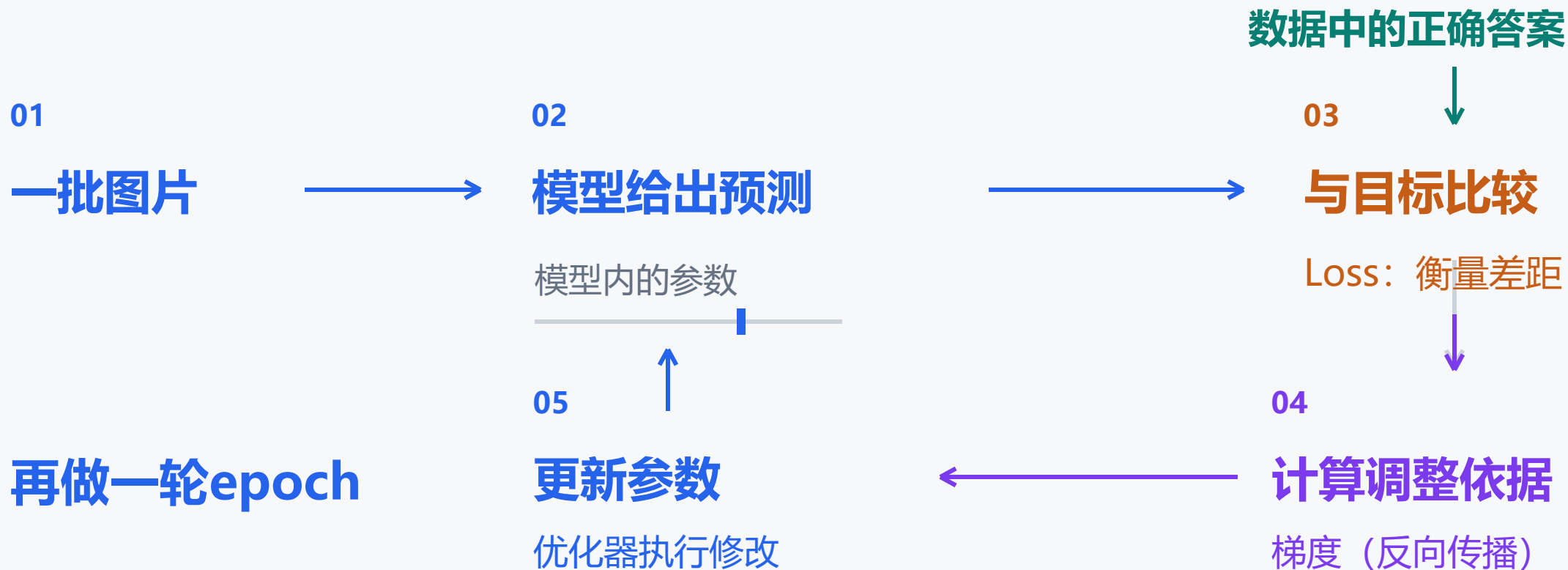
当前预测：芬达

真实类别：可乐

改哪个参数？ 往哪边改？ 改多少？

# 一轮训练的全貌

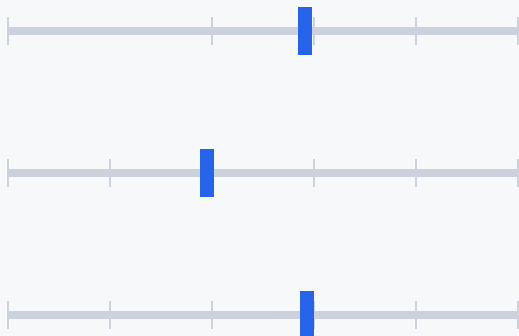
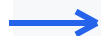
先预测，再比较，再算调整依据，最后改参数



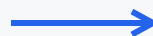
改变的是模型内部的参数，而不是超参数

# 参数改变预测分数

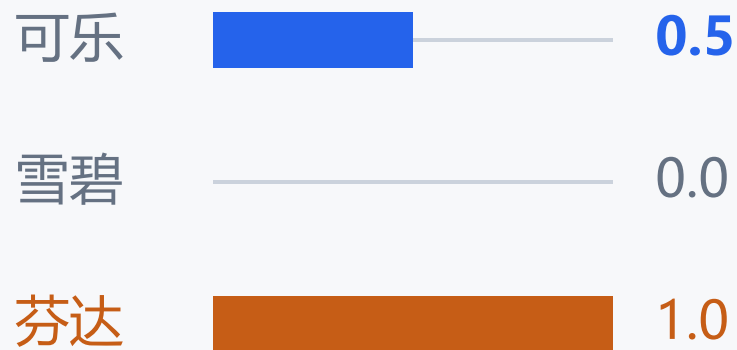
同一张图片，参数变了，预测分数也可能改变



权重、偏置



## 预测分数



仍然选芬达

训练通过数据调整参数

# Loss 记录改进程度

目标分数：可乐 1，雪碧 0，芬达 0

## 调整前

可乐 \_\_\_\_\_ 0.0

雪碧 \_\_\_\_\_ 0.0

芬达  1.0

## 仍选芬达

各项差距平方，再相加

$$1 + 0 + 1 = 2$$

  **Loss 2**

## 调整后

可乐  \_\_\_\_\_ 0.5

雪碧 \_\_\_\_\_ 0.0

芬达  1.0

## 仍选芬达

各项差距平方，再相加

$$0.25 + 0 + 1 = 1.25$$

  **Loss 1.25**

还没答对，也可以有所改进

# 从单张 Loss 到 batch Loss

同一组参数，分别给这一批图片评分



**Loss = 2**



**Loss = 1.25**



**Loss = 0.75**



**Loss = 4**

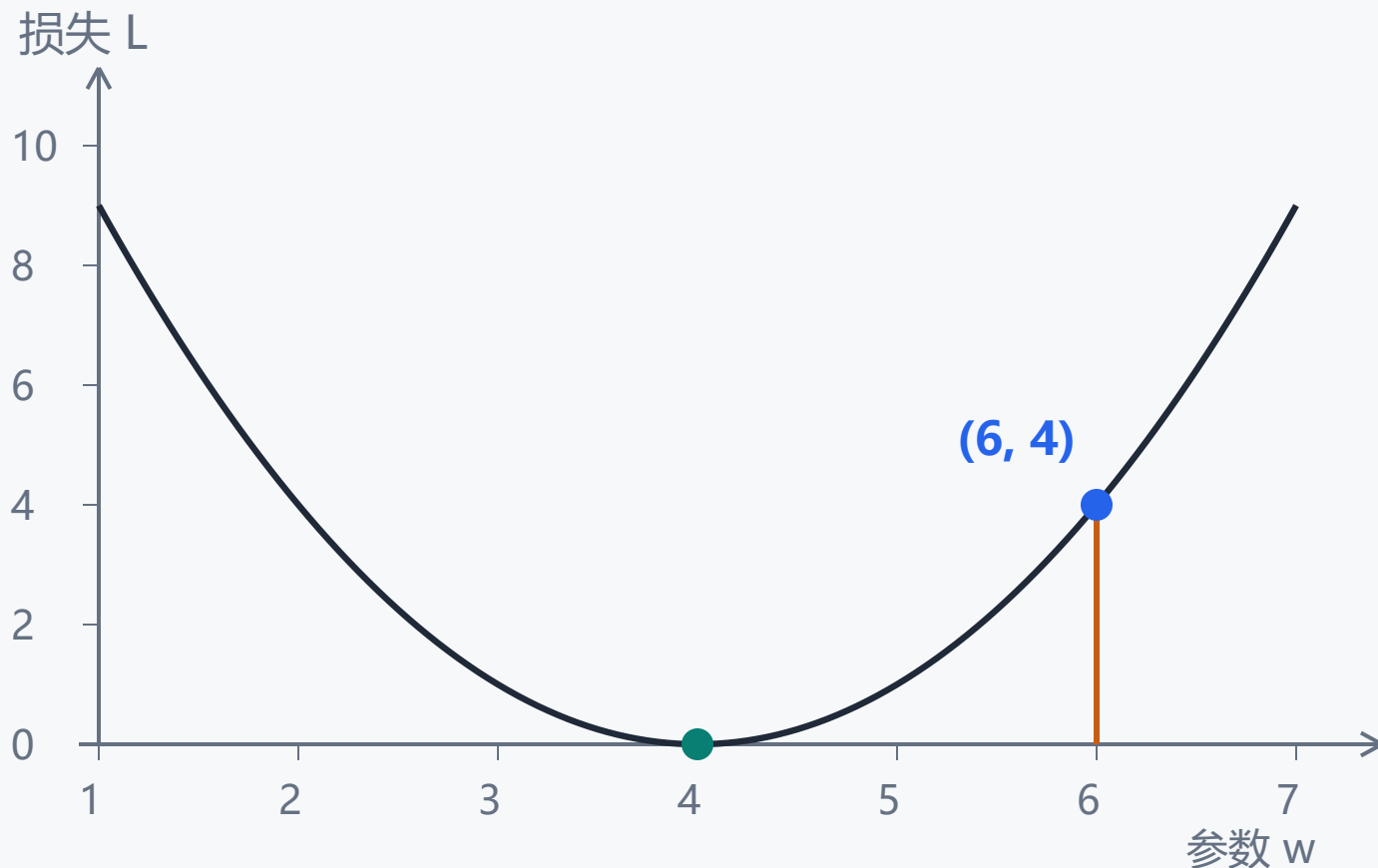
---

$$\text{batch Loss} = (2 + 1.25 + 0.75 + 4) \div 4 = 2$$

这 4 张图片组成一个 batch, batch size = 4

# 单参数的损失地图

暂时固定其他参数，观察一个参数的局部影响



当前位置

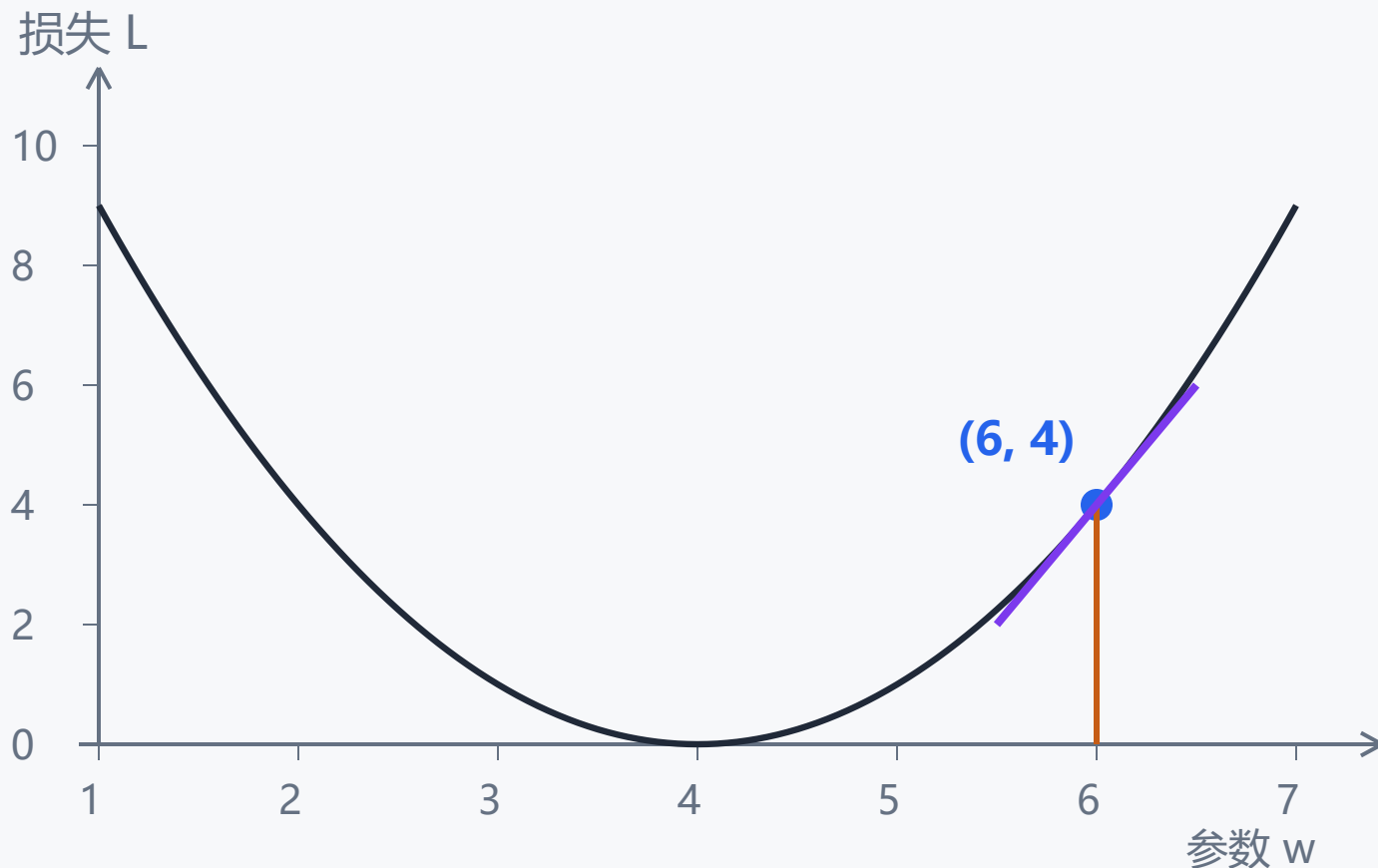
$$w = 6$$

$$L = 4$$

参数的位置，决定这个例子的损失高度

# 当前损失与局部坡度

同一个位置，两种不同的信息



高度和坡度，描述不同的信息

**Loss = 4**

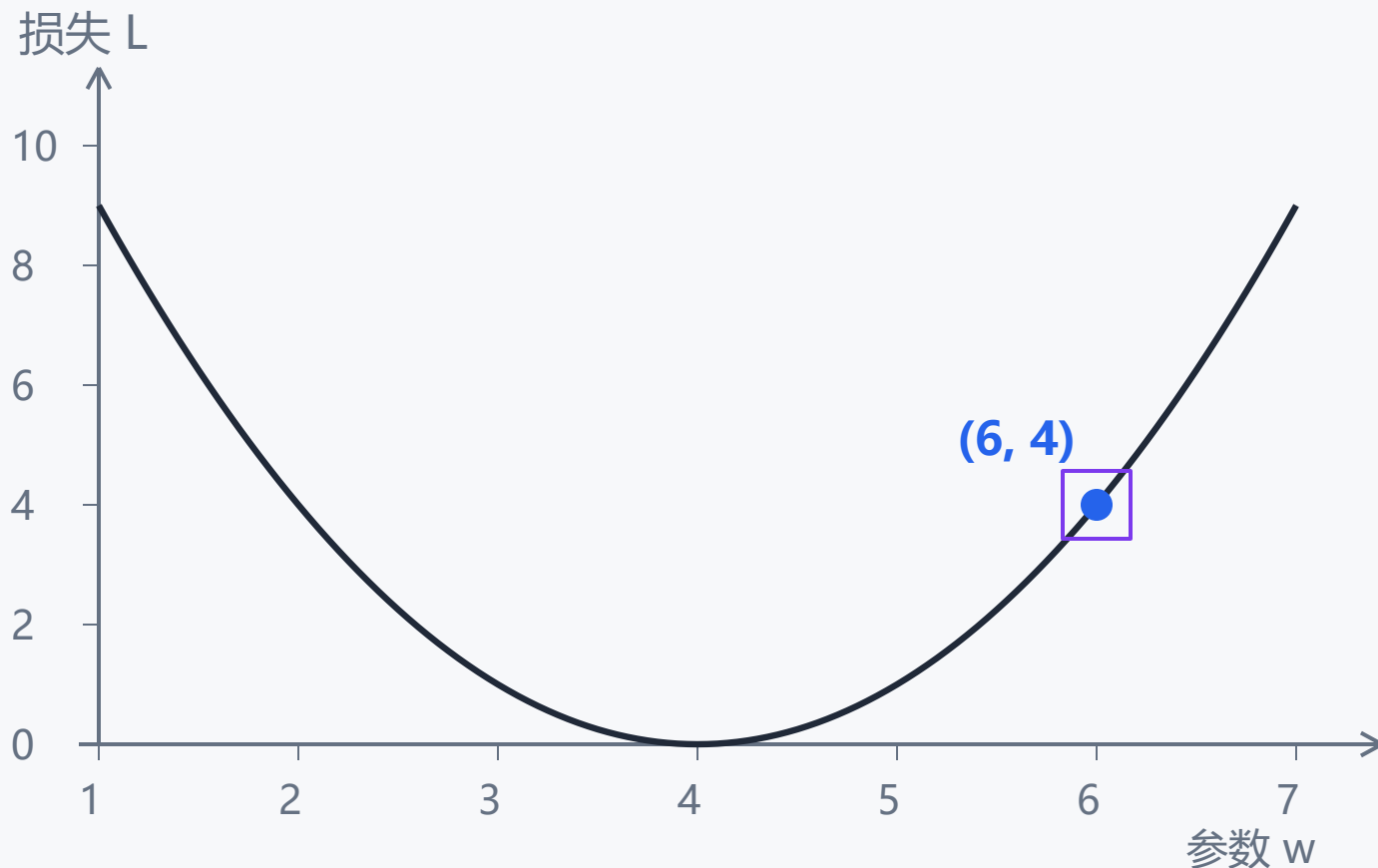
现在的错误评分

**导数 = +4**

往右微调，Loss 局部上升

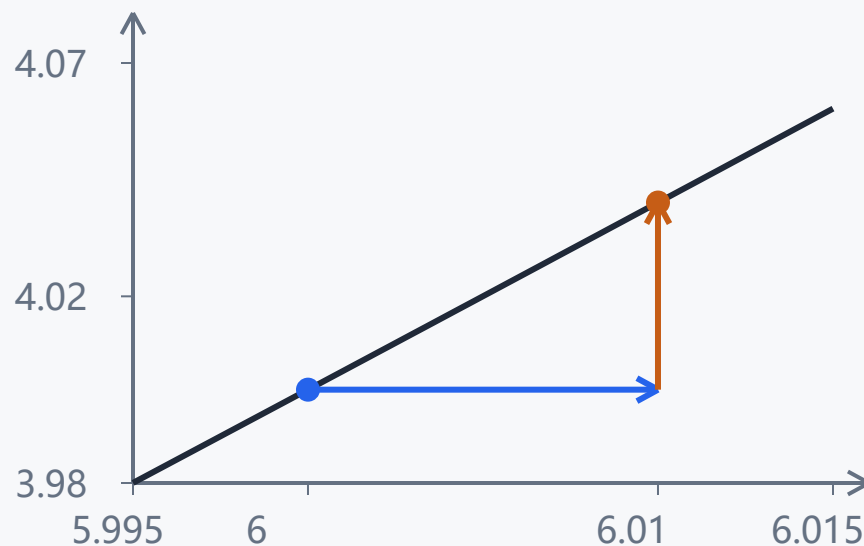
# 导数的局部含义

在  $w = 6$  附近，轻轻向右移动一点



当前位置，往左小调有望降低 Loss

局部放大

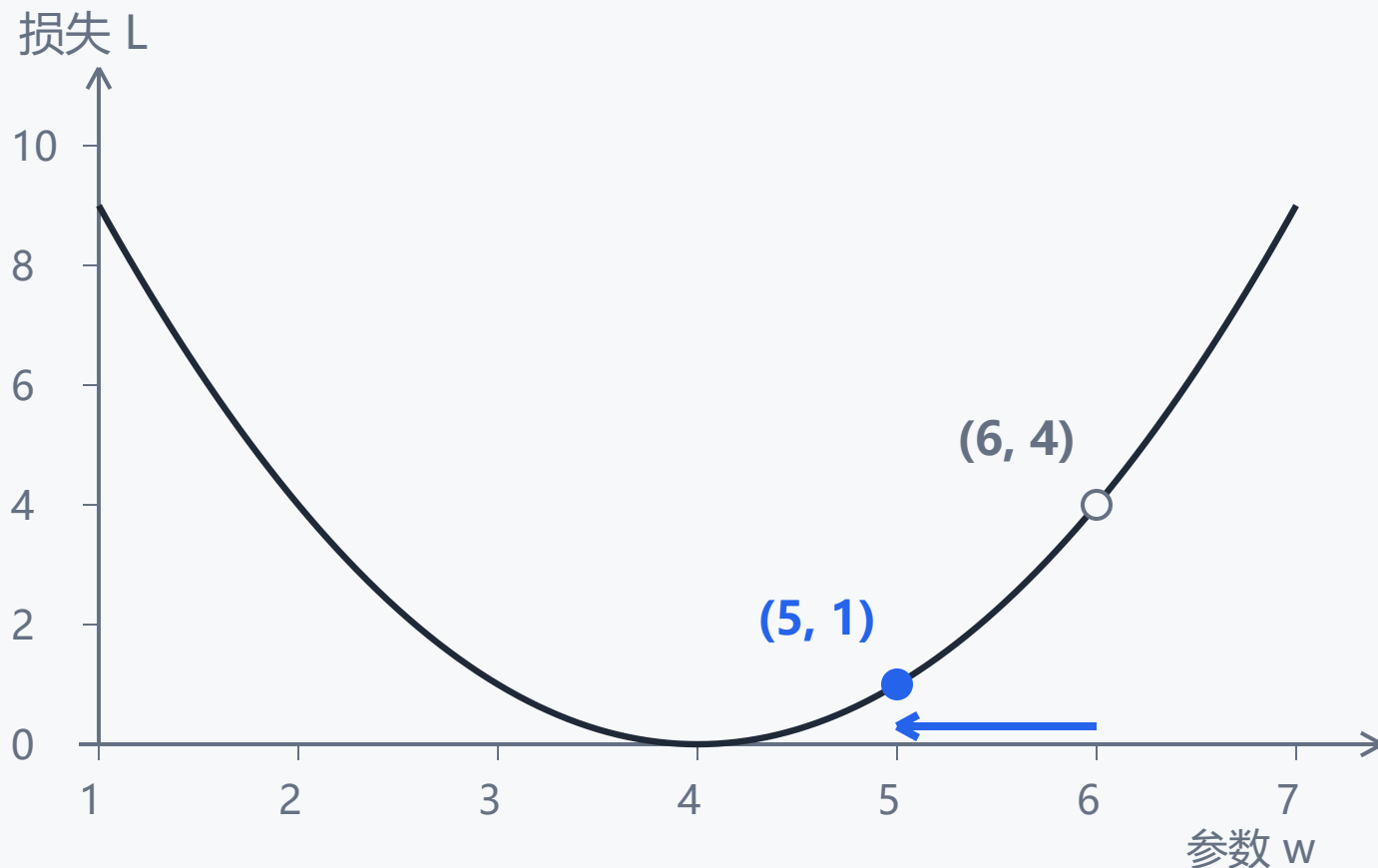


$w$  增加 0.01

$$\Delta L \approx 4 \times 0.01 = 0.04$$

# 一次参数更新

从  $w = 6$  出发, 学习率  $\eta = 0.25$



新参数 = 旧参数 - 学习率  $\times$  梯度

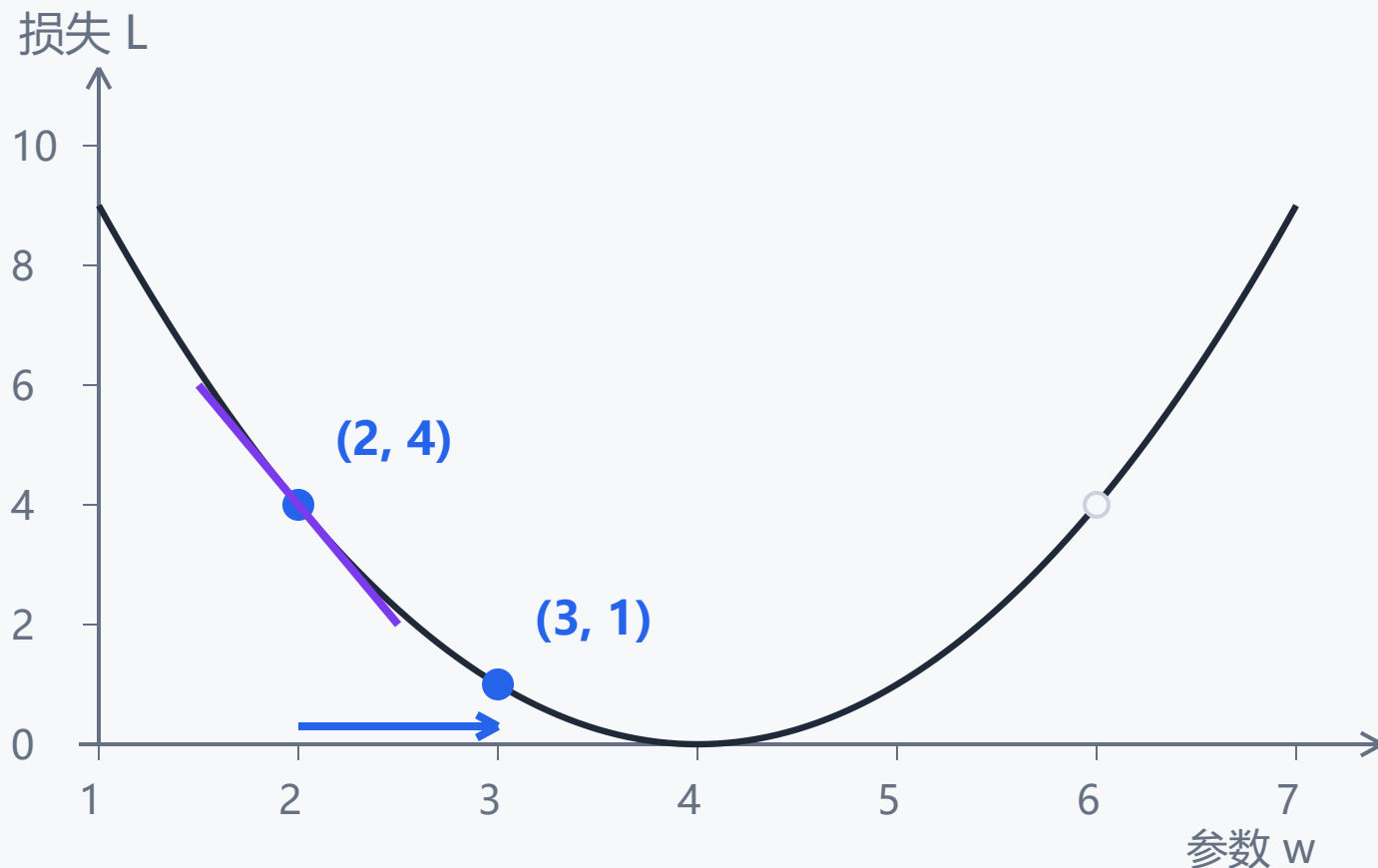
$$6 - 0.25 \times 4 = 5$$

Loss: 4 降到 1

走到新位置后, 需要重新计算梯度

# 换一个起点，参数怎么改？

如果改从  $w = 2$  出发？



$$\text{导数} = -4$$

$$2 - 0.25 \times (-4) = 3$$

Loss: 4 降到 1

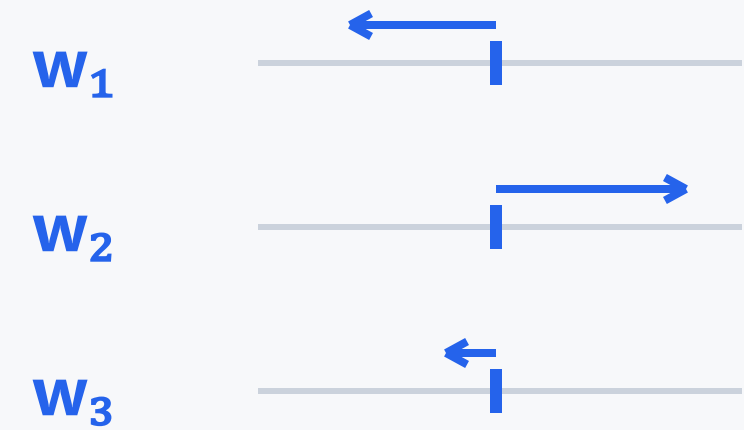
参数增大，也可以让损失降低

# 多个参数的梯度

暂时固定其他参数，观察一个参数的局部影响

$\eta = 0.1$

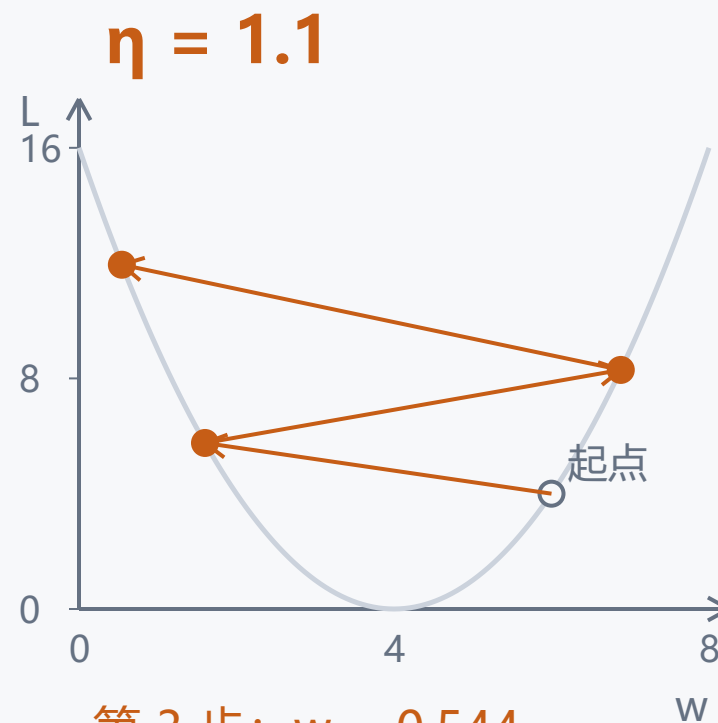
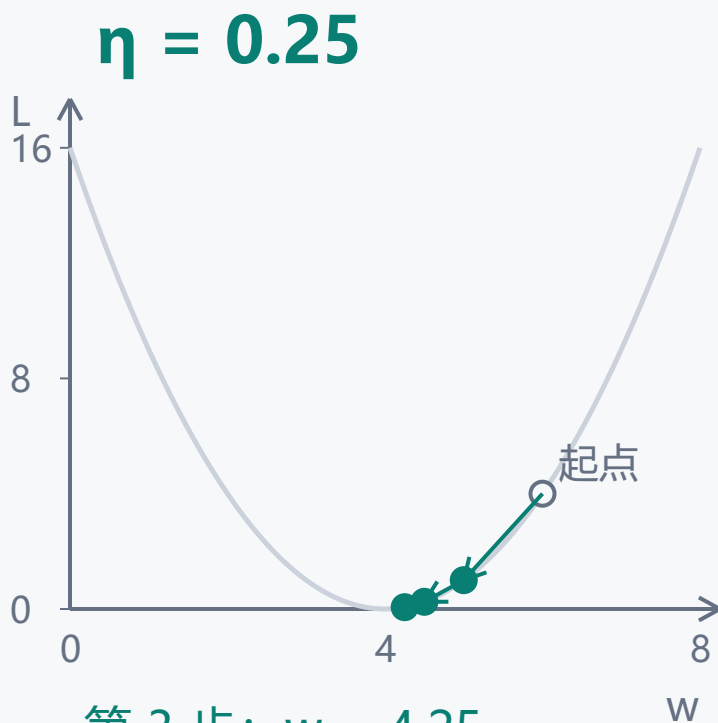
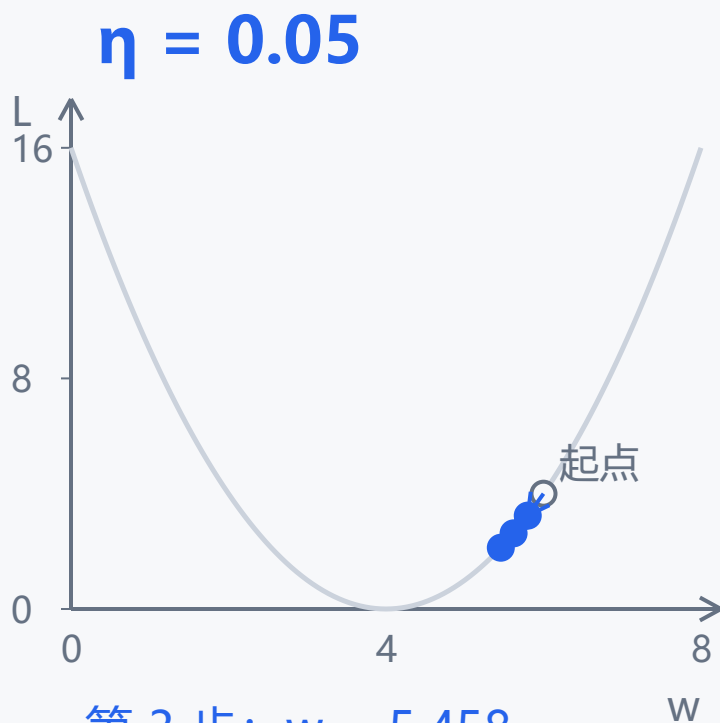
| 参数    | 当前梯度 | 本次改变量 |
|-------|------|-------|
| $w_1$ | +2   | -0.20 |
| $w_2$ | -3   | +0.30 |
| $w_3$ | +0.1 | -0.01 |



梯度是一组局部变化率

# 学习率改变更新幅度

同一函数  $L = (w - 4)^2$ , 同一起点  $w = 6$



步长由学习率和当前梯度共同决定

# 梯度怎样算出来？

逐个试探太麻烦，反向传播复用计算结果

## 逐个试探

每改一个参数，再评估

微调  $w_1$        $\longrightarrow$     重新计算 Loss

微调  $w_2$        $\longrightarrow$     重新计算 Loss

微调  $w_3$        $\longrightarrow$     重新计算 Loss

## 得到各参数的梯度

## 反向传播

保存中间结果，复用计算



沿计算关系回传，复用结果

# 链式法则的传递倍率

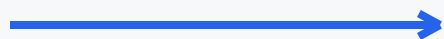
沿一条计算路径，观察微小变化怎样传递

起点

$\delta$

参数的微调

$\times 3$

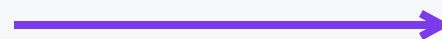


中间

$3\delta$

输出的变化

$\times 4$



终点

$\approx 12\delta$

Loss 的变化

沿一条路径，局部倍率相乘

# 前向计算

从输入出发，逐步算出中间值、输出和 Loss



Loss 已经算出来了，怎么知道哪些参数要改？

# 从 Loss 开始，计算第一个梯度

前向算数值，反向算局部敏感度



**$w$  动一点点,  $L$  怎么变怎么变?**

# 继续反向：复用已有的结果

沿原图继续向前，已算出的敏感度可以复用



还没有。权重仍然是 2 和 3

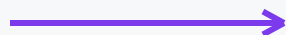
# 计算梯度与更新参数

反向传播得梯度，优化器根据梯度修改权重

反向传播：算梯度

$w_1$  的梯度：12

$w_2$  的梯度：8



梯度下降：更新参数

$\eta = 0.01$

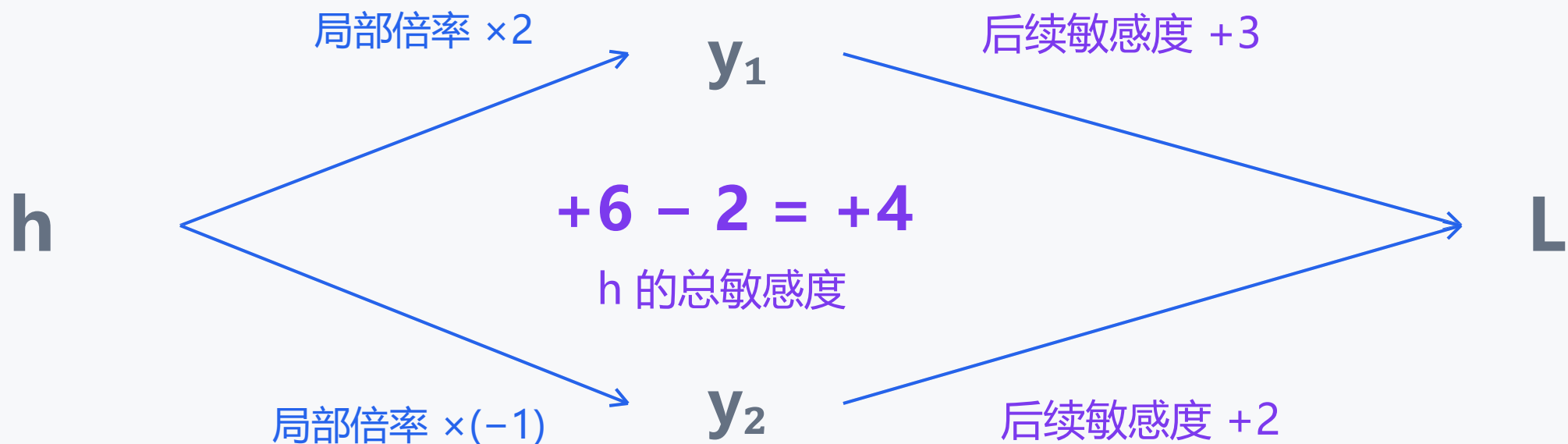
$w_1 = 2 \longrightarrow 1.88$

$w_2 = 3 \longrightarrow 2.92$

先算梯度，再由优化器修改参数

# 分叉路径的影响相加

加入一条分支：同一个中间值，影响两个输出



沿路径相乘，多条路径贡献相加

# 看多少张图片，才更新一次？

同样 8 张图片、同样顺序，按普通小批量训练比较

batch size

1



8 次更新

batch size

4



2 次更新

batch size

8



1 次更新

凑够一批，才计算这一批的梯度并更新一次

# 一批样本，共同决定调整方向

同一参数、同一旧状态，不同图片的梯度可能不同



梯度 +4



梯度 -2



梯度 +2



梯度 0

$$\text{batch 梯度} = (4 - 2 + 2 + 0) \div 4 = 1$$

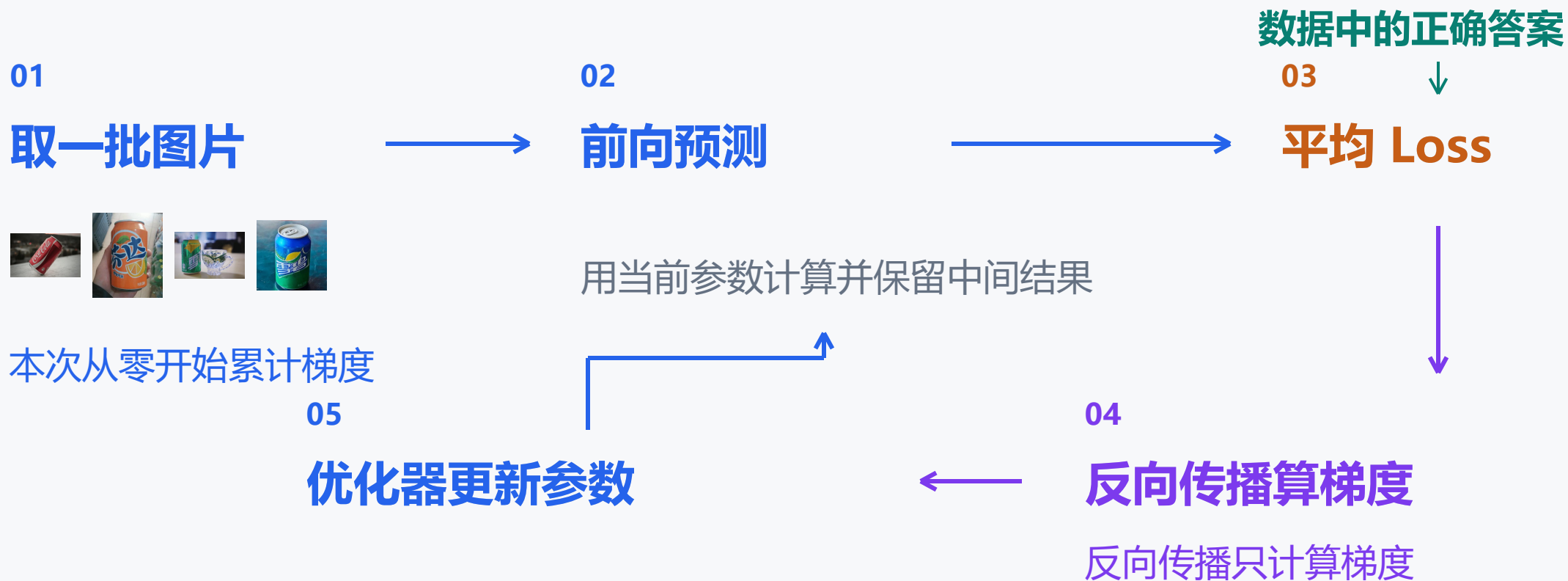
$\eta = 0.1$  时，这个参数的改变量为 -0.1

更多样本：梯度估计通常更稳定，但通常占用更多显存

**batch 更大，不代表训练一定更快或效果更好**

# 一次 step，完整发生了什么

从取出一个 batch，到完成一次参数更新



这一批的梯度用完，下一批从更新后的参数开始

# step 和 epoch

假设训练集只有这 8 张图片，batch size = 4



batch 1: 用  $\theta_0$ , 更新得到  $\theta_1$

batch 2: 用  $\theta_1$ , 更新得到  $\theta_2$

8 张都参与过一次: 1 个 epoch, 共 2 个 step

下个 epoch 通常重新打乱顺序, 从  $\theta_2$  继续

重新分批, 不会把模型参数重置回起点